

MOBL-99: Best Practice Summary

Series: MOBL — Mobile Monitoring | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/02/2026

Overview

This notebook consolidates every actionable best practice from the MOBL series (notebooks 01 through 12) into a single, definitive reference. Each best practice specifies the exact setting or value to use, its priority level, and which category it belongs to. Use this as a checklist when instrumenting, configuring, and operating Dynatrace native mobile monitoring.

Table of Contents

- 1. [SDK Setup & Configuration](#)
- 2. [Crash Reporting & Symbolication](#)
- 3. [User Action Instrumentation](#)
- 4. [Network Request Monitoring](#)
- 5. [Session Replay & Privacy](#)
- 6. [Session Properties & User Tagging](#)
- 7. [Privacy & Compliance](#)
- 8. [Dashboards & Alerting](#)
- 9. [SDK Performance Optimization](#)
- 10. [Advanced Instrumentation](#)
- 11. [DQL Query Patterns](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled

Requirement	Details
Permissions	<code>storage:user.events:read</code> , <code>storage:user.sessions:read</code> , <code>storage:smartscape:read</code> ; <code>storage:bizevents:read</code> for custom business events
Mobile App	At least one mobile application with Dynatrace SDK integrated
Prior Knowledge	Familiarity with MOBL-01 through MOBL-12 recommended

1. SDK Setup & Configuration

Best practices for installing and configuring the Dynatrace mobile SDK across iOS, Android, and cross-platform frameworks.

#	Best Practice	Recommended Setting/Value	Priority	Source
0	Turn on the New Real User Monitoring Experience	Environment: Settings > Collect and capture > Real User Monitoring > Enablement and cost control > Mobile > Enable RUM ; frontend: Settings > Enablement and cost control > New Real User Monitoring Experience . Without it the <code>user.events</code> / <code>user.sessions</code> queries return nothing	Critical	MOBL-02, MOBL-03
1	Enable auto-start for production apps	iOS: <code>DTXAutoStart = true</code> in <code>Info.plist</code> ; Android: <code>autoStart { applicationId(...); beaconUrl(...)} inside a named configuration in the top-level Gradle build file</code>	Critical	MOBL-02, MOBL-03
2	Keep crash reporting on	iOS: <code>DTXCrashReportingEnabled = true</code> ; Android: on by default -- do not set <code>crashReporting(false)</code>	Critical	MOBL-02, MOBL-03
3	Use Swift Package Manager over CocoaPods for iOS	Add via Xcode: File > Add Package Dependencies	Recommended	MOBL-02
4	Control the Gradle plugin version deliberately	Docs recommend <code>version "8.+"</code> in the top-level <code>plugins {}</code> block (minor updates automatic, majors by hand); pin an exact <code>8.x.y</code> if you need reproducible builds	Critical	MOBL-03
5	Fill in both platform sections of a cross-platform config	Use the Android and iOS values the instrumentation wizard gives you for each section	Critical	MOBL-04

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Use separate Application IDs for prod vs staging	Android: variant-specific configurations matched with <code>variantFilter</code> ; iOS: different Info.plist per scheme	Recommended	MOBL-03
7	Never commit Application IDs or Beacon URLs to public repos	Store in environment variables or <code>local.properties</code> excluded from version control	Critical	MOBL-03
8	Enable hybrid monitoring for WebView apps	iOS: <code>DTXHybridApplication = true ;</code> Android: <code>hybridWebView { enabled(true) }</code>	Recommended	MOBL-02, MOBL-03
9	For Flutter: use the downloaded <code>dynatrace.config.yaml</code> at project root	Apply it with <code>dart run dynatrace_flutter_plugin</code> , and start with <code>Dynatrace().start(MyApp())</code>	Critical	MOBL-04
10	For React Native: re-run <code>npx instrumentDynatrace</code> after config changes	Changing <code>dynatrace.config.js</code> does not auto-apply; the instrument command must re-run	Critical	MOBL-04
11	Verify both platforms report for cross-platform apps	Group <code>user.sessions</code> by <code>os.name</code> and confirm both Android and iOS sessions appear; list frontends with <code>smartscapeNodes "FRONTEND" \ filter frontend.type == "mobile" .</code> Classic equivalent: <code>fetch dt.entity.mobile_application -- not dt.entity.device_application ,</code> which does not exist and returns zero rows	Critical	MOBL-04

2. Crash Reporting & Symbolication

Best practices for ensuring crash data is complete, readable, and actionable.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Upload dSYM files for every iOS release build	Preprocess with <code>DTXDssClient -decode</code> , then <code>upload (DTXDssClient -upload</code> , Mobile Symbolication API, Fastlane plugin, or web UI)	Critical	MOBL-06

#	Best Practice	Recommended Setting/Value	Priority	Source
2	Upload ProGuard/R8 mapping files for every Android release build	Not automatic in the Gradle build: use the DSSClient, the Mobile Symbolication API (<code>DssFileManagement</code> token), the Fastlane plugin, or the web UI — from CI	Critical	MOBL-03, MOBL-06
3	Upload React Native source maps	Settings > Web and mobile monitoring > Source maps and symbol files > React Native > Upload files (bundle file, name, version)	Critical	MOBL-04
4	Verify symbolication status after upload	Settings > Web and mobile monitoring > Source maps and symbol files; pin files you must keep (1 GiB quota); stack traces must show method names, not hex addresses	Critical	MOBL-06
5	For Bitcode iOS builds: download dSYMs from App Store Connect	Local dSYMs do not match Apple-recompiled binaries; download from Xcode Organizer	Critical	MOBL-06
6	Leave OneAgent's R8 keep rules intact	They ship with the agent and R8 applies them; configure a third-party obfuscator to honor them, and do not filter them out (<code>ignoreFrom</code>)	Critical	MOBL-03
7	Report handled exceptions via SDK API	iOS: <code>DTXAction.reportError(withName:error:)</code> ; Android: <code>Dynatrace.reportError(name, exception)</code>	Recommended	MOBL-06
8	Use descriptive error names for reported errors	"Payment Failed" not "Error" -- enables meaningful DQL grouping	Recommended	MOBL-06
9	Report errors at catch boundaries, not in utility methods	Attach full context (user action, screen name) where the error is handled	Recommended	MOBL-06
10	Do not report transient/expected conditions as errors	Network retries, expected timeouts, and recoverable conditions create noise	Recommended	MOBL-12
11	Set a crash-free target from your baseline	Community starting bands: > 99.5% healthy, 98-99.5% review, < 98% act; Google Play's user-perceived crash threshold is 1.09% (per user, not per session)	Critical	MOBL-10, MOBL-11

3. User Action Instrumentation

Best practices for tracking user interactions accurately and meaningfully.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Instrument SwiftUI with the SwiftUI instrumentor	<code>brew install DTSwiftInstrumentor</code> , then <code>DTSwiftInstrumentor install</code> ; add it to CI builds	Critical	MOBL-02, MOBL-05
2	Rely on Jetpack Compose auto-instrumentation	On by default from Android Gradle plugin 8.271; add <code>enterAction</code> / <code>leaveAction</code> only for business-level flows	Critical	MOBL-03, MOBL-05
3	Always call <code>leaveAction()</code> in a finally block	Forgetting to close an action inflates duration metrics and orphans child events	Critical	MOBL-03, MOBL-05
4	Use descriptive, stable action names	"Cart: Add Item" not "btn_click" or "button_42"	Critical	MOBL-05
5	Never put dynamic values in action names	"View Product Details" not "View Product #48291" -- dynamic values cause cardinality explosion	Critical	MOBL-05
6	Never put user IDs or timestamps in action names	Creates infinite unique names, defeats grouping, and is a privacy risk	Critical	MOBL-05
7	Use feature-area prefixes in naming convention	"Cart: Add Item", "Search: Apply Filter", "Checkout: Submit Order"	Recommended	MOBL-05
8	Use consistent casing across platforms	"Search Products" on both iOS and Android, not mixed case variants	Recommended	MOBL-05
9	Exclude SwiftUI controls that add noise	The SwiftUI instrumentor supports global and local exclusion of controls	Recommended	MOBL-02
10	Configure user action naming rules in Dynatrace UI	Mobile app settings > Naming rules (naming or extraction rules on the generated name); test against recent data before deploying	Recommended	MOBL-05
11	Keep action hierarchies shallow	Parent + one level of child actions (the Flutter plugin allows no deeper)	Recommended	MOBL-05, MOBL-12

4. Network Request Monitoring

Best practices for HTTP monitoring, backend correlation, and network performance optimization.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Instrument backend services with OneAgent or OpenTelemetry	Required for <code>x-dynatrace</code> / W3C trace-context correlation (W3C from OneAgent for Mobile 8.333); enables end-to-end distributed tracing from device to database	Critical	MOBL-07
2	Use manual web request tagging for non-standard transports	Add the header named by <code>Dynatrace.getRequestTagHeader()</code> (<code>x-dynatrace</code>) with a tag from <code>action?.getTagFor(url)</code> (iOS) or <code>action.getRequestTag()</code> / <code>Dynatrace.getRequestTag()</code> (Android) for WebSocket, gRPC, or custom HTTP clients	Recommended	MOBL-12
3	Enable gzip/brotli compression on API responses	Substantially smaller text payloads	Recommended	MOBL-07
4	Implement response pagination	Avoid loading entire datasets in a single request	Recommended	MOBL-07
5	Use HTTP/2 or HTTP/3 for multiplexing	Reduces connection overhead and round trips	Recommended	MOBL-07
6	Implement client-side caching with ETags and Cache-Control	Prevents redundant network requests for unchanged resources	Recommended	MOBL-07
7	Serve static assets from a CDN	Reduces DNS, TCP, and TLS times for geographically distributed users	Recommended	MOBL-07
8	Adapt app behavior to connection type	Reduce image quality, defer non-critical requests, enable offline queueing on slow connections	Optional	MOBL-07
9	When the backend span dominates a request's <code>duration</code> , investigate the backend	Use frontend-to-backend correlation to trace into server-side services and database calls; mobile request events carry no DNS/TCP/TLS breakdown	Recommended	MOBL-07
10	Exclude analytics/CDN URLs from SDK monitoring	iOS <code>DTXURLFilters</code> in Info.plist; Android <code>urlFilters</code> in the plugin's <code>webRequests</code> block (8.339+)	Recommended	MOBL-12

5. Session Replay & Privacy

Best practices for configuring mobile session replay with appropriate privacy controls.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Enable session replay on crash	App settings > General > Enablement and cost control > Enable Session Replay on crashes (no client-side key)	Critical	MOBL-08
2	Choose the masking level deliberately	Safest is the default; to use Safe, set <code>MaskingConfiguration(maskingLevelType: .safe)</code> (iOS) or <code>MaskingConfiguration.Safe()</code> (Android) in code	Critical	MOBL-08
3	Set the production Full Session Replay percentage to 5-10%	App settings > General > Enablement and cost control; there is no SDK sample rate	Critical	MOBL-08
4	Set staging/QA sample rate to 100%	Full coverage for pre-release testing and bug verification	Recommended	MOBL-08
5	Use Safest masking for financial/healthcare apps	Replays show layout and navigation but no readable content	Critical	MOBL-08
6	Only use Custom masking when you need to unmask specific elements	Start with Safest or Safe, move to Custom only for targeted debugging; mask views via <code>MaskingConfiguration (addMaskedView / addMaskedIds)</code> , <code>Modifier.dtMask()</code> in Compose, or the <code>data-dtrum-mask</code> tag	Recommended	MOBL-08
7	Temporarily increase sample rate during incidents	Raise to 50-100% when actively investigating a reported issue; lower again after resolution	Recommended	MOBL-08
8	Monitor Session Replay consumption	Track it in the license overview (DEM units on classic licensing; your DPS rate card otherwise)	Recommended	MOBL-08
9	Limit what session replay captures (masking, sampling)	Replay retention is fixed at 35 days in a built-in bucket that cannot currently be modified	Optional	MOBL-09

6. Session Properties & User Tagging

Best practices for enriching sessions with business context and identifying users.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Tag users with opaque IDs, never PII	<code>Dynatrace.identifyUser("usr_a1b2c3d4")</code> -- never email addresses, phone numbers, or full names	Critical	MOBL-09
2	Clear user tag on logout	iOS: <code>Dynatrace.identifyUser(nil)</code> ; Android: <code>Dynatrace.identifyUser(null)</code>	Critical	MOBL-09
3	Report session-property values early in the session	Call <code>action.reportValue()</code> on an early user action (values must be part of a user action)	Recommended	MOBL-09
4	Keep the property set small and deliberate	Every property is data you send, store and must justify under data minimization	Recommended	MOBL-09
5	Use enum-like string values for properties	<code>"tier" = "free"</code> not <code>"tier" = "Free Trial Account"</code> -- enables clean aggregation	Recommended	MOBL-09
6	Use consistent property key names across iOS and Android	Same keys on both platforms so a single DQL query covers both	Recommended	MOBL-09
7	Never store PII in session property values	A property is stored as sent — treat it like any field you would have to delete on request	Critical	MOBL-09
8	Report A/B test variants as session properties	<code>action.reportValue("ab_test_checkout_v2", "variant_b")</code> on an open action -- enables correlation with performance/crash data	Recommended	MOBL-12

7. Privacy & Compliance

Best practices for GDPR, CCPA, and general data privacy compliance.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Implement opt-in mode for GDPR compliance	<code>DTXUserOptIn = true</code> (iOS) or <code>userOptIn(true)</code> (Android); call <code>applyUserPrivacyOptions</code> after user consent	Critical	MOBL-09
2	Collect nothing until the user chooses	In opt-in mode OneAgent captures no data until <code>applyUserPrivacyOptions</code> is called; <code>PERFORMANCE</code> is the anonymous level a user can choose, <code>USER_BEHAVIOR</code> the full one	Critical	MOBL-09
3	Rely on OneAgent to persist the applied preferences	OneAgent persists privacy preferences and re-applies them on restart; the app only needs to ask when no choice exists yet	Critical	MOBL-09
4	Provide a way to withdraw consent in app settings	Set data collection level to <code>off</code> when user revokes consent	Critical	MOBL-09
5	Handle right-to-erasure requests with the documented deletion routes	Sensitive Data Center deletion request (UI) or Grail record deletion (API, covers <code>user.events</code> , <code>user.sessions</code> , <code>user.replays</code>)	Critical	MOBL-09
6	Minimize retained personal data	RUM data sits in built-in 35-day buckets that cannot currently be modified; set short retention only on data you route yourself (business events) and minimize what the SDK captures	Recommended	MOBL-09
7	Separate crash reporting opt-in from general monitoring	Crash reporting has its own <code>crashReportingOptedIn</code> flag independent of data collection level	Recommended	MOBL-09
8	Display clear, plain-language consent dialog	Explain what data is collected, why, and how; provide granular options for Performance vs User Behavior levels	Critical	MOBL-09
9	Do not plan per-region RUM buckets	Custom RUM buckets cannot currently be created, so RUM data cannot be split into EU and non-EU buckets with OpenPipeline	Optional	MOBL-09

8. Dashboards & Alerting

Best practices for operationalizing mobile monitoring with dashboards and alerts.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Include a crash-free rate single-value tile with color thresholds	Community starting bands: green > 99.5%, yellow 98-99.5%, red < 98% — tune to your baseline	Critical	MOBL-11
2	Organize dashboard: health at top, trends in middle, drill-down at bottom	Top: single-value tiles (crash rate, sessions); Middle: timeseries (trends); Bottom: tables (top crashes, slowest actions)	Recommended	MOBL-11
3	Add a dashboard variable for <code>frontend.name</code>	Lets stakeholders filter to their specific app	Recommended	MOBL-11
4	Set default time range to 24 hours with presets for 1h, 6h, 24h, 7d	Covers most operational use cases without excessive data scanning	Recommended	MOBL-11
5	Alert on high crash rate	Davis anomaly detector on a DQL crash query (<code>characteristics.has_crash</code>), static threshold such as > 10 crashes per hour	Critical	MOBL-11
6	Alert on session volume drop	< 50% of rolling 7-day baseline, severity Warning	Recommended	MOBL-11
7	Alert on slow app launch	Average app start > 5 seconds over a 15-minute window, severity Warning	Recommended	MOBL-11
8	Alert on high HTTP error rate	> 5% of mobile requests returning 5xx, severity Critical	Critical	MOBL-11
9	Use Dynatrace Intelligence for adaptive baselines on performance metrics	Dynatrace Intelligence automatically learns patterns; add static thresholds only for hard SLA limits	Recommended	MOBL-11
10	Tag mobile app entities with <code>app-type: mobile</code>	Enables the Problem trigger's affected-entity tag filter for mobile-specific problems (or use the custom filter <code>matchesValue(affected_entity_ids, "MOBILE_APPLICATION- *")</code>)	Recommended	MOBL-11
11	Create audience-specific dashboards	Developers: 5-min refresh, crash detail; QA: 15-min, crash-free rate by version; Executives: daily KPI snapshot	Recommended	MOBL-11
12	Break down crash rate by <code>app.short_version</code> after every release	Identifies if a specific release introduced a regression	Critical	MOBL-10, MOBL-11

9. SDK Performance Optimization

Best practices for minimizing SDK overhead on battery, network, and CPU.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Exclude third-party analytics and CDN URLs from monitoring	iOS <code>DTXURLFilters</code> (Info.plist array, wildcards); Android <code>urlFilters</code> in <code>webRequests</code> (8.339+)	Recommended	MOBL-12
2	Use cost and traffic control for high-traffic production apps	App settings > General > Enablement and cost control; a lower monitored-session share reduces DEM consumption	Recommended	MOBL-08, MOBL-12
3	Start with full monitoring, reduce only if needed	Do not prematurely optimize; measure SDK overhead first	Recommended	MOBL-12
4	Set an app-launch target from your baseline	Community rule-of-thumb bands: < 1s excellent, 1-2s acceptable, 2-5s needs work, > 5s investigate; re-baseline after OneAgent for Mobile 8.347 caps inflated cold starts	Critical	MOBL-10, MOBL-11

10. Advanced Instrumentation

Best practices for custom business events, A/B testing, multi-app strategies, and advanced SDK usage.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use reverse-domain naming for custom business event types	<code>"com.myapp.purchase"</code> not <code>"purchase"</code> -- avoids collisions across apps and services	Recommended	MOBL-12
2	Send business events for conversion-critical interactions	Purchases, sign-ups, feature usage via <code>sendBizEvent()</code> with structured attributes	Recommended	MOBL-12
3	Report feature flag evaluations as business events	<code>event.type = "com.myapp.feature_flag"</code> with <code>feature.name</code> and <code>feature.variant</code> attributes	Optional	MOBL-12
4	Use nested actions for multi-step flows	Parent action wraps the full flow; child actions measure individual steps (validate, submit, confirm)	Optional	MOBL-12

#	Best Practice	Recommended Setting/Value	Priority	Source
5	For long-running actions, manage lifecycle explicitly	Call <code>leaveAction()</code> in completion handler or <code>finally</code> block; do not rely on auto-close timeout	Critical	MOBL-12
6	Use consistent attribute names across iOS and Android business events	Same keys on both platforms enable unified DQL queries	Recommended	MOBL-12
7	Never include PII in business event attributes	No email addresses, phone numbers, or personal data in custom event payloads	Critical	MOBL-12
8	Use consistent naming prefixes for multi-app organizations	"MyBrand Customer", "MyBrand Driver", "MyBrand Admin" for easy DQL filtering	Recommended	MOBL-12
9	Budget DEM units per app based on criticality	100% monitoring for critical apps; use cost and traffic control for high-traffic secondary apps	Recommended	MOBL-12
10	Attach <code>reportValue()</code> properties to actions for business context	<code>action?.reportValue(withName: "total_price", doubleValue: 49.99) --</code> convert to action/session properties in the app settings	Recommended	MOBL-12

11. DQL Query Patterns

Mandatory patterns and filters for querying mobile data in Grail.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Query mobile RUM from <code>user.events / user.sessions</code>	<code>filter dt.rum.application.type == "mobile"</code> separates mobile from web; <code>characteristics.has_*</code> selects the event type. Mobile RUM is not in <code>bizevents</code>	Critical	MOBL-10
2	Count sessions on <code>user.sessions</code>	One record per session, so <code>count()</code> is a session count there; on <code>user.events</code> use <code>countDistinct(dt.rum.session.id)</code>	Critical	MOBL-10
3	Compute crash rate at session grain	<code>fetch user.sessions then countIf(error.has_crash == true) over count()</code>	Recommended	MOBL-10

#	Best Practice	Recommended Setting/Value	Priority	Source
4	Add <code>isNotNull()</code> filters before grouping on optional fields	Fields like <code>app.short_version</code> , <code>device.model.identifier</code> , <code>device.manufacturer</code> may be null; filter first to avoid noise	Recommended	MOBL-10
5	Query mobile app entities for inventory	Preferred: <code>smartscapeNodes "FRONTEND" \ filter frontend.type == "mobile"</code> . Classic fallback (still functional): <code>fetch dt.entity.mobile_application</code> . No time range needed; returns current state. <code>dt.entity.device_application</code> does not exist -- it returns zero rows, not an error	Recommended	MOBL-10
6	Filter crashes with <code>characteristics.has_crash</code>	Reported errors are <code>characteristics.has_error</code> with <code>characteristics.is_api_reported</code> -- distinguish between the two	Critical	MOBL-06, MOBL-10
7	Always specify explicit time ranges	<code>fetch user.events, from:-1h</code> or <code>from:-24h</code> -- never rely on the default 2-hour window	Critical	MOBL-10
8	Segment by <code>app.short_version</code> for release monitoring	Enables crash rate and performance comparison across releases	Recommended	MOBL-10

Priority Summary

Priority	Count	Description
Critical	47	Must implement -- failure to do so causes data loss, security risk, broken monitoring, or compliance violation
Recommended	52	Should implement -- significantly improves data quality, operational efficiency, or user experience
Optional	5	Nice to have -- advanced capabilities for mature mobile monitoring programs

Implementation Order

1. **First:** All Critical items in SDK Setup, Crash Reporting, and Privacy sections
2. **Second:** Critical items in User Action Instrumentation and Dashboards/Alerting

3. **Third:** All Recommended items
 4. **Last:** Optional items for advanced use cases
-

References

- [Dynatrace Mobile Monitoring Documentation](#)
 - [iOS SDK Integration Guide](#)
 - [Instrument Android apps \(DT docs\)](#)
 - [Session Replay for Mobile](#)
 - [Mobile SDK Privacy Settings](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.